

INTERNSHIP REPORT

On

AI-Powered URL Shortener (NeoLnk)

Submitted To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student: 2410030664

Name of Student: SAURAV RAJ TIWARY

Batch: 2026-27

CANDIDATE'S DECLARATION

I, Saurav Raj Tiwary, do hereby declare that the internship report titled "AI-Powered URL Shortener (NeoLnk)" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Saurav Raj Tiwary

Roll No.: 2410030664

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to thank Mr. Kaushlendra Singh Sisodia, Chief Mentor at The IoT Academy and Director of Uniconverge Technologies, and the entire upSkill Campus and AICTE team, for providing me the opportunity and platform to undertake this internship and for their continuous mentorship throughout the program.

I express my gratitude to my parents for their blessings.

Signature

Student Name: Saurav Raj Tiwary

Roll No.: 2410030664

INTERNSHIP COMPLETION CERTIFICATE



TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	
1.	Candidate's Declaration	
2.	Acknowledgment	
3.	Internship Completion Certificate	
4.	Project Description	
	4.1 Introduction	
	4.2 Organization Profile	
	4.3 Problem Statement	
	4.4 Project Objectives	
	4.5 Scope of the Project	
	4.6 Technologies and Tools Used	
	4.7 System Architecture	
	4.8 Methodology	
	4.9 Testing and Verification	
	4.10 Expected Outcomes / Results	
5.	Bibliography/References	

4. Project Description

4.1 Introduction

This report documents the design, development, and verification of an AI-powered URL Shortener application named "NeoLnk," undertaken as part of a Python-focused internship conducted under the guidance of The IoT Academy, Uniconverge Technologies, and upSkill Campus, in collaboration with the All India Council for Technical Education (AICTE). The project involved building a complete web-based URL shortening service with a Python Flask backend, a SQLite database, and a modern, animated front-end dashboard for link management and analytics.

URL shortening services are widely used to convert long, unwieldy web addresses into short, shareable links, while also enabling click tracking and analytics. NeoLnk was built to demonstrate proficiency in backend API design, database schema design, and frontend visualization, while incorporating practical features such as custom aliases, link expiration, QR code generation, and real-time analytics dashboards.

4.2 Organization Profile

The internship program was conducted in collaboration with the All India Council for Technical Education (AICTE), Uniconverge Technologies, and upSkill Campus. Uniconverge Technologies is an IoT and technology solutions provider that, through The IoT Academy, offers industry-oriented training programs designed to bridge the gap between academic learning and real-world technical skills. upSkill Campus partners with AICTE to provide structured internship and skilling programs to engineering students across India, with a focus on hands-on, project-based learning in domains such as Python, IoT, and data science.

4.3 Problem Statement

Long URLs are difficult to share, remember, and track. Organizations and individuals often need a reliable way to shorten links, brand them with custom aliases, control their validity period through expiration settings, and monitor how those links are being used (number of clicks, browsers, and platforms of visitors). Off-the-shelf shortening services often lack transparency, customization, and self-hosted control over data. The

project therefore addresses the need for a self-contained, lightweight, and analytics-capable URL shortening platform that can be deployed and controlled independently.

4.4 Project Objectives

- To design and implement a Flask-based backend capable of generating unique, collision-free short codes using a Base62 encoding scheme.
- To allow users to create custom aliases for shortened links, with proper collision handling.
- To support link expiration constraints so that shortened URLs can be set to expire automatically.
- To build a redirection engine that logs every click along with User-Agent-derived browser and platform information.
- To develop a visually engaging, responsive front-end dashboard for creating, listing, and managing shortened links.
- To provide real-time analytics through interactive charts, including click history and browser/OS breakdowns.
- To validate the system through both automated unit testing and manual end-to-end browser verification.

4.5 Scope of the Project

The scope of the project covers the complete lifecycle of a shortened URL: creation (with optional custom alias and expiration), storage, redirection, and analytics. It includes a REST API layer, a persistent SQLite data store, and a client-side dashboard with QR code generation and charting. The project is scoped as a self-hosted, single-server application suitable for demonstration, internal organizational use, or as a foundation for a production-grade link management service. Advanced features such as user authentication, multi-tenant support, and horizontal scaling were considered outside the scope of this internship phase.

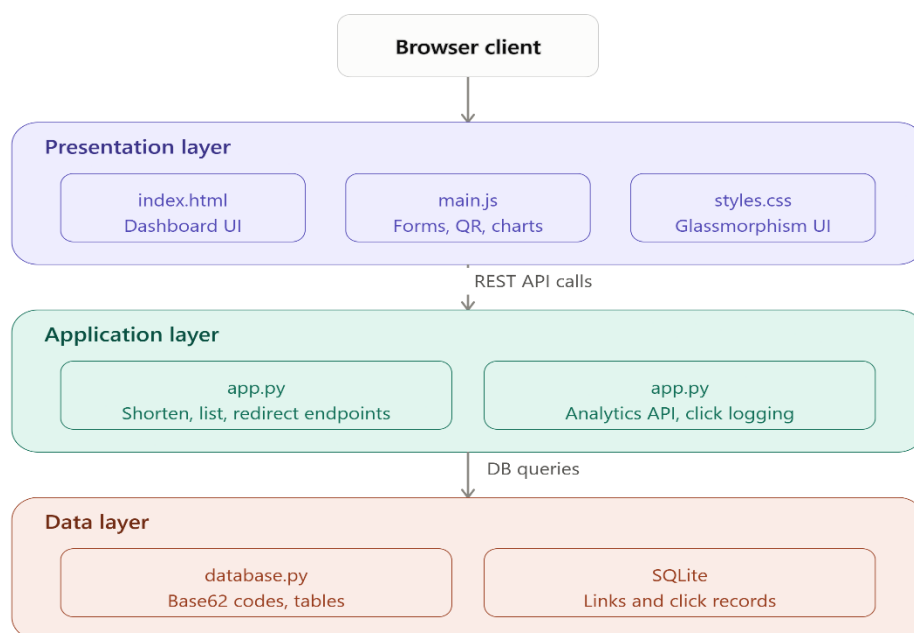
4.6 Technologies and Tools Used

- Backend: Python, Flask
- Database: SQLite

- Backend Libraries: Flask, user-agents (for parsing browser/platform information from click requests)
- Frontend: HTML5, Vanilla CSS (glassmorphism design), JavaScript
- Frontend Libraries: Chart.js (doughnut and line charts for analytics), QRCode.js (client-side QR code rendering)
- Typography: Outfit Google Font
- Testing: Python standard library unit testing framework
- Tooling: PowerShell (test execution), browser-based manual verification

4.7 System Architecture

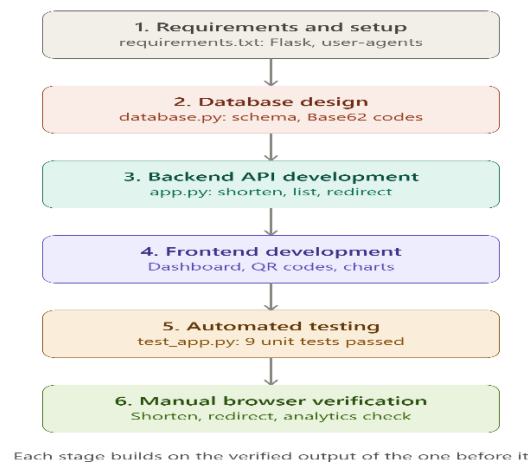
NeoLnk follows a classic three-tier web application architecture. The presentation layer consists of a single-page dashboard (index.html, styles.css, main.js) that communicates with the backend over REST API calls. The application layer is implemented in app.py, which exposes endpoint controllers for shortening transactions, list retrieval, live analytics payloads, and asynchronous click-tracking redirections. The data layer, implemented in database.py, manages the SQLite connection, table schemas, Base62 uniqueness logic, analytics aggregation functions, and click recording. When a shortened link is visited, the redirection endpoint logs the click event (capturing browser and platform via User-Agent parsing) before forwarding the visitor to the destination URL, and this data feeds the analytics dashboard in near real time.



4.8 Methodology

The project was developed and verified in the following stages:

- Requirements and dependency setup: Configuring requirements.txt with Flask and the user-agents library.
- Database design: Building database.py with SQLite table setup, Base62-based unique short-code generation, and analytics aggregator and click-recording functions.
- Backend API development: Implementing app.py with endpoints for URL shortening, listing existing links, serving analytics data, and handling redirection with click tracking.
- Frontend development: Building index.html with a glassmorphism-styled dashboard, styles.css for neon grid templates and animations, and main.js for form handling, toast notifications, QR code rendering, and Chart.js-based analytics visualizations.
- Automated testing: Writing and executing unit tests in test_app.py covering Base62 validation, custom alias collision handling, expiration constraints, User-Agent parsing, REST API input/status-code handling, and redirection behaviour.
- Manual verification: Performing end-to-end browser-based testing of link generation, QR code display, copy-to-clipboard functionality, redirection, and analytics chart rendering.



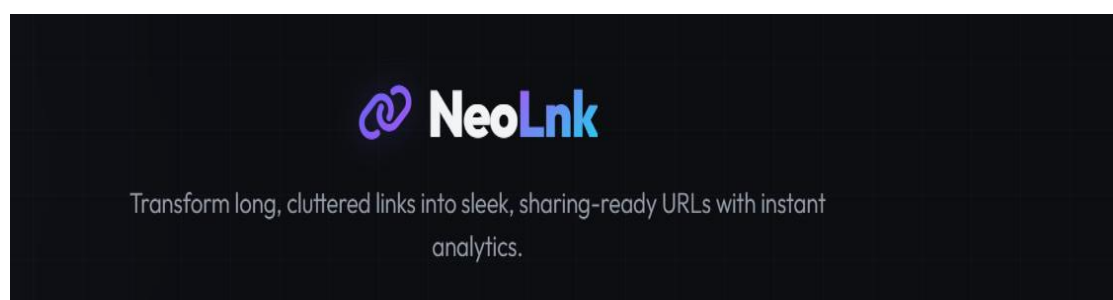
4.9 Testing and Verification

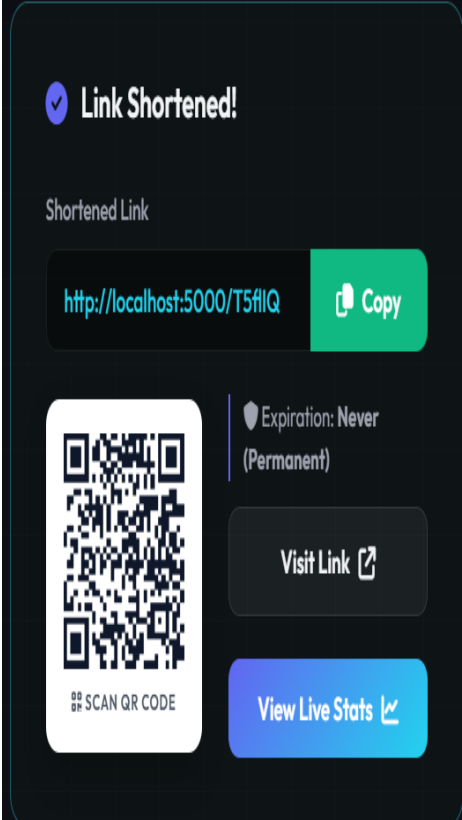
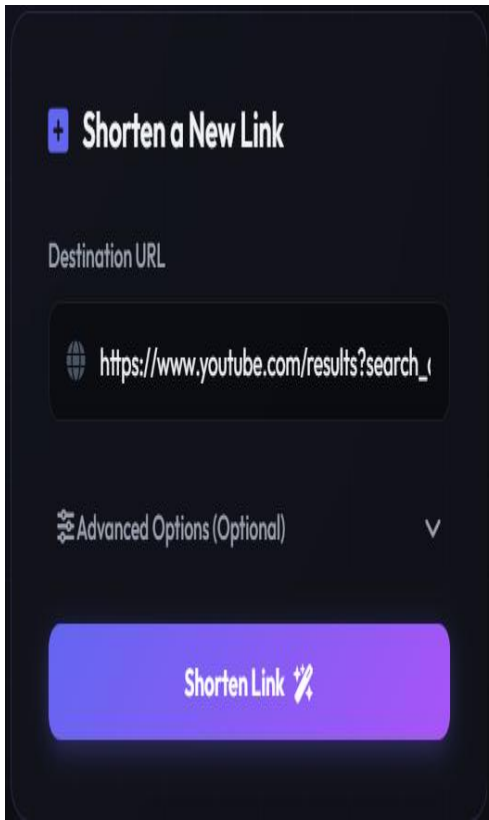
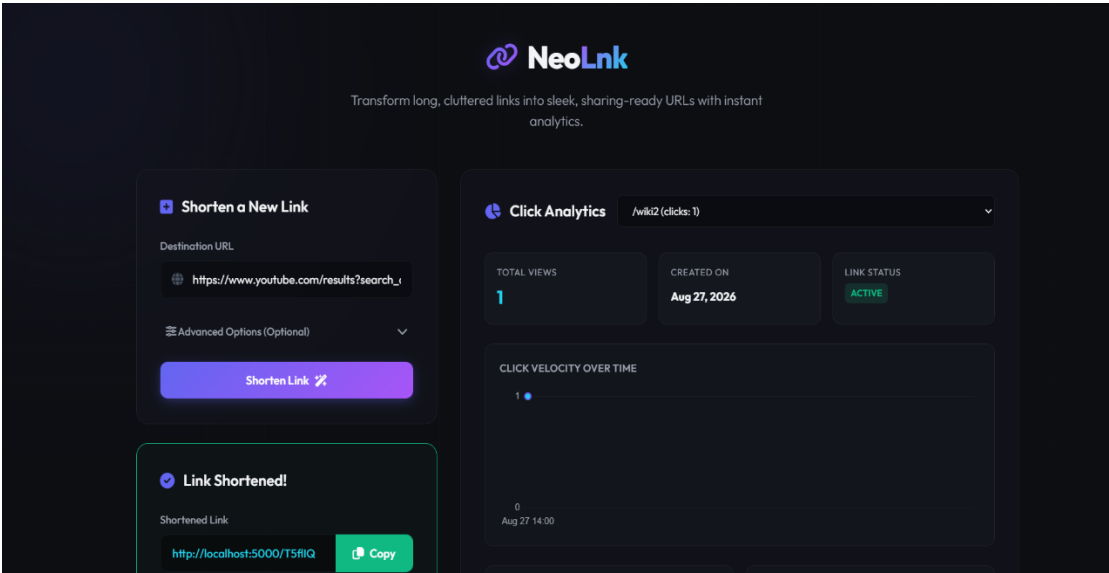
Automated integration tests were implemented in `test_app.py` using Python's standard library testing tools and executed via the command line. All nine automated test cases passed successfully, covering Base62 validation, custom alias collision handling, expiration constraints, User-Agent-based browser and platform parsing, REST API input validation and status codes, and redirection behaviour.

Manual browser verification was subsequently performed to validate the complete user flow. A destination URL (<https://wikipedia.org>) was shortened using the custom alias "wiki2," and the resulting short link (<http://localhost:5000/wiki2>) was confirmed to work correctly. The QR code was displayed instantly upon link creation, and the copy-to-clipboard function produced the expected toast notification. Visiting the shortened link correctly redirected to Wikipedia and incremented the click count to 1. The analytics dashboard was verified to correctly render the click-history line chart along with browser (Chrome) and operating-system breakdown charts, confirming that click events were being captured and visualized accurately.

4.10 Expected Outcomes / Results

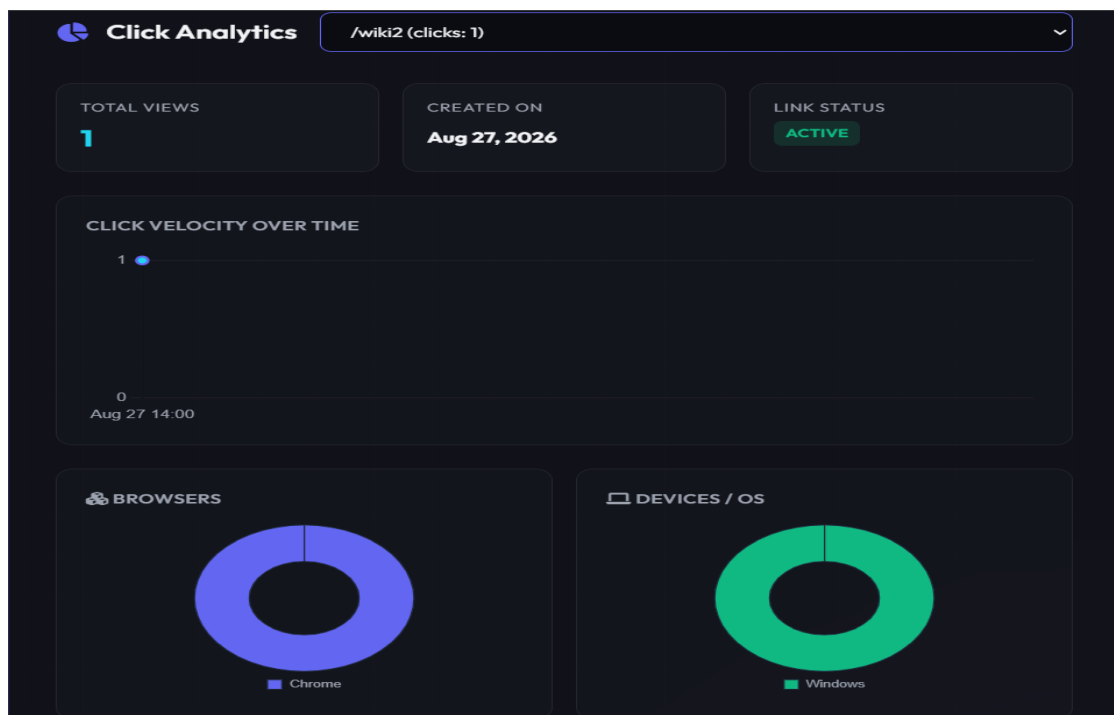
The completed NeoLnk application successfully meets all defined objectives. It reliably generates unique short links using Base62 encoding, supports custom aliases with proper collision handling, and enforces link expiration constraints. The redirection engine accurately tracks clicks along with visitor browser and platform information. The dashboard provides an intuitive, visually polished interface for managing links, complete with QR code generation and real-time analytics visualizations. All automated unit tests and manual verification scenarios passed, demonstrating that the system is functionally correct and ready for demonstration or further development toward production deployment.





Shortened Links Inventory Refresh

Short URL	Destination URL	Clicks	Created	Expiration	Action
T5fllQ	https://www.youtube.com/results?se...	0	8/27/2026	Never	Stats
wiki2	https://wikipedia.org	1	8/27/2026	8/28/2026	Stats



4.11 Certificate of Completion and Communication Proof: The Certificate of Completion issued by AICTE, Uniconverge Technologies, and upSkill Campus for this internship (Certificate ID: USC246792TIA), covering the period 02-08-2026 to 30-08-2026, is attached in Chapter 3 of this report as proof of successful completion.

5. Bibliography/References

- [1] Flask Documentation. Pallets Projects. Available at:
<https://flask.palletsprojects.com/>
- [2] Python Software Foundation. Python 3 Documentation. Available at:
<https://docs.python.org/3/>
- [3] SQLite Documentation. Available at: <https://www.sqlite.org/docs.html>
- [4] Chart.js Documentation. Available at: <https://www.chartjs.org/docs/latest/>
- [5] QRCode.js Library. Available at: <https://github.com/davidshimjs/qrcodejs>
- [6] AICTE, Uniconverge Technologies, and upSkill Campus - Internship Program
Certificate of Completion, Certificate ID: USC246792TIA.